



나만의 운동 도우미 웹앱

Python Week 프로젝트 발표

왜 GymHelper를 만들었나요?

운동할 때 흩어져 있는 필요한 기능들을 한 곳에



무게를 얼마나 들어야 할지 계산이 필요함 (1RM)



오늘 무슨 운동을 몇 kg, 몇 회 했는지 기록이 필요함



세트 사이 휴식 시간을 재는 타이머가 필요함



다음엔 얼마나 더 들어야 할지 코칭이 필요함



그리고... 자세가 맞는지 확인해줄 사람이 없음

GymHelper 주요 기능

5가지 기능을 카드 형태로 제공



1RM 계산기

최대 중량 계산



운동 기록

운동 저장 및 조회



운동 타이머

세트 자동 관리



AI 운동 코치

맞춤 루틴 추천



스쿼트 자세 분석

웹캠 실시간 자세 체크

웹사이트는 어떻게 만들어졌나

Flask 하나로 5개 기능을 라우팅 + Render 자동 배포

/

홈

기능 5개를 카드로 보여주는 첫 화면

/onerm

1RM 계산기

폼으로 무게·횟수 입력 → 계산식으로 최대 중량 산출

/record

운동 기록

입력값을 data/workout.csv 파일에 저장하고 목록으로 조회

/timer

운동 타이머

서버 로직 없이 순수 프론트엔드 JS로 카운트다운 진행

/coach

AI 코치

입력값 기준으로 규칙 기반 추천 문구 생성

/squat

자세 분석

페이지만 서버가 내려주고, 분석은 브라우저 JS가 전담

배포 흐름

GitHub에 push → Render가 자동 감지해 `pip install -r requirements.txt` 실행 → `gunicorn app:app`으로 서비스 시작

기능 살펴보기: 스쿼트 자세 분석

웹캠으로 실시간 자세 체크 + 피드백

목표

- ✓ 카메라 앞에서 스쿼트를 하면 자동으로 반복 횟수를 세어준다
- ✓ 얼마나 깊이 앉았는지(깊이), 얼마나 일정한 속도로 했는지 분석한다
- ✓ 세트가 끝나면 바로 개선할 점을 문장으로 알려준다

예시 피드백 문구

"무릎이 덜 굽혀졌어요. 허벅지가 바닥과 평행이 되도록 더 깊이 앉아보세요."

"속도가 빨라요. 내려갈 때 2~3초, 올라올 때 1~2초로 천천히 해보세요."

1차 시도: OpenCV를 그대로 웹사이트에 합치기

cv.py (로컬 파이썬 스크립트)를 Flask 서버에 그대로 연결하려 함

cv.py 구조

```
cv2.VideoCapture(0)          # 내 컴퓨터 웹캠을 직접 읽기
mediapipe (python)          # 파이썬으로 포즈 인식
cv2.imshow(...)              # 화면에 창을 띄워서 결과 표시
cv2.waitKey('s' / 'q')       # 키보드로 세트 종료/전체 종료
```

❌ 시도 실패 — OpenCV 방식은 서버에 그대로 올라가지 않았습니다

- ❌ 서버(Render)에는 웹캠이 없다 → cv2.VideoCapture(0)가 읽을 카메라 자체가 없음
- ❌ 서버에는 화면(GUI)이 없어서 cv2.imshow 창을 띄울 수 없다
- ❌ 키보드 입력도 서버가 아니라 '내 브라우저'에서 눌러야 한다

그래서 방향을 바꿨습니다

마감 안에 끝내기 위해, 같은 로직을 브라우저 JS로 다시 구현

cv.py (Before · Python)

- cv2.VideoCapture(0) → 로컬 웹캠 읽기
- mediapipe (python 패키지) → 자세 인식
- cv2.imshow(...) → 로컬 GUI 창에 출력
- cv2.circle / cv2.putText → 프레임에 그리기
- cv2.waitKey('s'/'q') → 키보드 입력

squat.js (After · Browser JS)

- getUserMedia() → 방문자 브라우저 웹캠
- @mediapipe/tasks-vision (JS/WASM) → 자세 인식
- <canvas> → 브라우저 화면에 직접 출력
- ctx.arc / ctx.fillText → 캔버스에 그리기
- 버튼 클릭 이벤트 → 세트 종료 / 전체 종료

각도 계산 · 반복 판정 상태머신 · 피드백 조건 같은 '분석 로직' 자체는 그대로 두고,
'영상 입출력을 담당하던 OpenCV 부분'만 브라우저 표준 기술로 통째로 교체했습니다.

동작 원리

각도 계산 → 상태 판정 → 반복 카운트 → 피드백

1

관절 좌표 추출

엉덩이·무릎·발목, 3개 관절 좌표를 매 프레임 추정

2

각도 계산

세 점 사이 각도를 계산해 지금 얼마나 굽혔는지 측정

3

상태 판정

서있음 → 내려감 → 올라옴, 상태 변화로 1회 반복 인식

4

세트 분석

평균 깊이·편차·속도를 계산해 피드백 문장 생성

기술 스택

프론트엔드 (브라우저)

- MediaPipe Tasks Vision (JS)
- WebAssembly (WASM)
- getUserMedia / Canvas API
- ES Modules (JavaScript)

백엔드 / 배포

- Flask (Python)
- Gunicorn
- GitHub
- Render (자동 배포)

이번 프로젝트를 통해 배운 것



'로컬에서 되는 것'과 '서버에서 되는 것'은 다르다

카메라·GUI처럼 서버에 없는 자원은 클라이언트(브라우저)로 옮겨야 한다는 걸 직접 겪으며 배웠습니다.



같은 로직도 언어/환경을 바꿔 다시 구현할 수 있다

파이썬으로 짰 각도 계산·상태 판정 로직을 자바스크립트로 그대로 옮기는 연습을 했습니다.



협업 중인 저장소에 기능을 더하는 방법

collaborator 권한, fork, branch, Pull Request 같은 실제 협업 흐름을 처음부터 다뤄봤습니다.



Git → GitHub → Render로 이어지는 배포 파이프라인

코드를 push하면 자동으로 배포되는 과정을 직접 구성하고 확인했습니다.

감사합니다 🙌

배포 사이트: gymhelper-squat.onrender.com